ST0244 — LENGUAJES DE PROGRAMACIÓN · EAFIT

Guía de Estudio

Práctica I — Sistema de Registro Universitario

Haskell (Versión A) · Prolog (Versión B)

Para la defensa presencial — Este documento cubre exactamente lo que evalúa la guía: operación correcta de ambas implementaciones (30%) y capacidad de explicar estructuras de datos, manejo de archivos, cálculo de tiempo y diferencias entre paradigmas (70%).

Requerimientos Funcionales — Qué hace cada opción

La guía exige **5 funciones idénticas** en ambas versiones. Esta es la tabla de referencia rápida:

#	Opción	Qué pide la guía	Cómo lo implementamos
1	**Check In**	Pedir ID, registrar hora de entrada	Valida que no esté ya adentro, guarda en archivo
2	**Search by ID**	Buscar estudiante activo, mostrar info o mensaje	`find` en lista / unificación en Prolog
3	**Time Calculation**	Calcular duración con entrada y salida, mostrar claramente	Resta de minutos, formatea como `HH:MM hs`
4	**List Students**	Cargar `University.txt` → almacenar en lista → mostrar	Recarga desde disco en el momento de elegir la opción
5	**Check Out**	Pedir ID, registrar hora de salida, actualizar datos	Actualiza lista y guarda en `University.txt`

Haskell — Versión A

El tipo de dato `Estudiante`

```haskell
data Estudiante = Estudiante {
    idEst   :: String,
    entrada :: Int,     -- Minutos desde 00:00 (ej: 08:30 = 510)
    salida  :: Int      -- -1 si aún está en campus
} deriving (Show, Read)
```

> **Para explicar en la defensa:** `deriving (Show, Read)` permite serializar/deserializar el dato directamente con `show` y `read`, lo que hace que la persistencia en archivo sea trivial — cada línea del archivo es exactamente la representación textual del dato.

Gestión del tiempo (Opción 2 de la guía)

La guía acepta tres formas. Usamos la **Opción 2**: el usuario ingresa `HH:MM` y lo almacenamos como entero de minutos.

```haskell
parsearHora "08:30" → Just 510
parsearHora "25:00" → Nothing   ← validación de rango
parsearHora "abc"   → Nothing   ← validación de formato
```

Cálculo de permanencia:

```
TiempoSalida - TiempoEntrada = Duración en minutos
740 - 510 = 230 minutos → formatearTiempo 230 → "03:50 hs"
```

> **Para explicar en la defensa:** Almacenar como entero simplifica el cálculo a una resta. Si usáramos strings "HH:MM" directamente, necesitaríamos parsear dos veces para calcular la diferencia.

Inmutabilidad y listas (estructura de datos)

La guía pide: *"Use a list to store student information in memory during execution"*

En Haskell **no existen variables mutables**. En lugar de modificar la lista, cada función devuelve una lista nueva:

```haskell
-- Check In: agrega al frente (O(1))
checkIn: Estudiante id t (-1) : lista

-- Check Out: recorre con map, reemplaza el registro correcto
registrarSalida id hSalida lista =
    map (\e -> if idEst e == id && salida e == -1
```

```
                then e { salida = hSalida }
                else e) lista
```

> **Para explicar en la defensa:** `e { salida = hSalida }` es *record update syntax* de Haskell — crea una copia del registro `e` con solo el campo `salida` modificado. No muta el original.

Persistencia en archivo

La guía pide: cargar al inicio, guardar en cada Check In / Check Out.

```haskell
-- Guardar: convierte cada Estudiante a String con show
guardarArchivo lista =
    writeFile "University.txt" (unlines (map show lista))

-- Cargar: crea el archivo si no existe, parsea cada línea con read
cargarArchivo = do
    existe <- doesFileExist "University.txt"
    if not existe
        then writeFile "University.txt" "" >> return []
        else do
            contenido <- readFile "University.txt"
            let lineas = filter (not . null) (lines contenido)
            return (map read lineas)
```

> **Punto importante para la defensa — Lazy Evaluation:** Haskell es perezoso. `readFile` no lee el archivo inmediatamente, lo hace cuando se accede al contenido. Por eso `filter (not . null) (lines contenido)` fuerza la evaluación antes de que el archivo pueda ser escrito de nuevo en el mismo ciclo.

Opción 4 — List Students (diferencia clave)

La guía dice literalmente: *"Choosing this option must: Load student information from University.txt → Store it in a list → Display the list"*

Nuestra implementación recarga desde disco al elegir esta opción:

```haskell
studentsList :: IO [Estudiante]
studentsList = do
    lista <- cargarArchivo        -- lee University.txt en este momento
    if null lista
        then putStrLn "La lista esta vacia."
        else mapM_ printFila lista
    return lista                  -- devuelve la lista para actualizar el menú
```

El menú recibe la lista actualizada: `"4" -> studentsList >>= menu`

Menú — Recursividad paramétrica

```haskell
menu :: [Estudiante] -> IO ()
menu lista = do
    -- ... mostrar opciones ...
    opcion <- getLine
    case opcion of
        "1" -> checkIn lista      >>= menu   -- nueva lista al menú
```

```
        "2" -> searchByID lista   >>  menu lista
        "3" -> timeCalculation lista >> menu lista
        "4" -> studentsList       >>= menu   -- recarga desde disco
        "5" -> checkOut lista     >>= menu
        "6" -> putStrLn "Hasta luego!"
        _   -> menu lista
```

Para explicar en la defensa: No hay `while` ni variables globales. El "estado" del programa (la lista de estudiantes) viaja como argumento de función en función. `>>=` (bind) pasa el resultado de IO a la siguiente función.

Prolog — Versión B

Base de conocimiento dinámica

La guía pide: *"Use a list (or a list of structures/terms) to store student information"*

En Prolog usamos **hechos dinámicos** como estructura de almacenamiento:

```
:- dynamic estudiante/3.
% estudiante(Id, Entrada, Salida)
% Salida = 0 → estudiante aún en campus

estudiante('12345678', 510, 0).
estudiante('87654321', 555, 740).
```

> **Para explicar en la defensa:** En Prolog no hay listas mutables en el sentido imperativo. Los hechos dinámicos *son* la estructura de datos — la base de conocimiento actúa como la lista en memoria que pide la guía.

Modificación de datos: `retract` + `assertz`

En Prolog "actualizar" un hecho significa eliminarlo y agregar uno nuevo:

```
registrar_salida :-
    read(Id),
    retract(estudiante(Id, Entrada, 0)),   % elimina el hecho anterior
    read(HoraSalida),
    assertz(estudiante(Id, Entrada, HoraSalida)),  % agrega el actualizado
    guardar_datos.
```

> **Contraste con Haskell:** Haskell genera una nueva lista con `map`. Prolog retracta el hecho viejo y aserta uno nuevo. Ambos logran el mismo resultado pero desde paradigmas opuestos: funcional vs. lógico.

Persistencia

```
% Cargar desde archivo
cargar_datos :-
    retractall(estudiante(_, _, _)),   % limpia la base
    open('University.txt', read, Flujo),
    leer_lineas(Flujo),                % lee términos con read/2
    close(Flujo).

% Guardar al archivo
guardar_datos :-
```

```
    open('University.txt', write, Flujo),
    forall(
        estudiante(Id, E, S),
        format(Flujo, 'estudiante(~w, ~w, ~w).~n', [Id, E, S])
    ),
    close(Flujo).
```

Menú con `repeat/0`

```
menu :-
    cargar_datos,
    repeat,                     % equivalente a un bucle infinito
        mostrar_opciones,
        read(Opcion),
        ejecutar(Opcion),
    Opcion == 5, !.             % el corte rompe el repeat cuando se elige salir
```

> **Para explicar en la defensa:** `repeat/0` siempre tiene éxito y genera puntos de retroceso infinitos. El corte `!` al verificar `Opcion == 5` elimina esos puntos de retroceso, terminando el ciclo. Es el equivalente lógico de un `while(!salir)`.

Diferencias entre Paradigmas

La guía evalúa específicamente: *"Differences between functional (Haskell) and logic programming (Prolog)"*

Aspecto	Haskell — Funcional	Prolog — Lógico
Estructura de datos	Lista `[Estudiante]` pasada como argumento	Hechos `estudiante/3` en base de conocimiento
Modificar un registro	`map` genera nueva lista	`retract` + `assertz` del hecho
Ciclo del menú	Recursividad paramétrica	`repeat/0` con corte `!`
Búsqueda	`find` con lambda sobre lista	Unificación de patrones directa
Persistencia — leer	`readFile` + `map read (lines ...)`	`read/2` término a término
Persistencia — escribir	`writeFile` + `map show`	`format/3` sobre flujo abierto
Flujo de control	Funciones puras + `IO` explícito	Predicados con efectos secundarios implícitos
Estado mutable	No existe — viaja en argumentos	Existe vía `assert/retract`

Posibles preguntas del docente

¿Por qué usaron minutos enteros para el tiempo? La guía lo permite explícitamente como Opción 2/3. Simplifica el cálculo de duración a una resta simple y evita el manejo de librerías de tiempo que varían entre entornos.

¿Por qué `salida = -1` en Haskell y `salida = 0` en Prolog? En Haskell `-1` es un valor imposible para un tiempo válido (los minutos son ≥ 0). En Prolog se usó `0` como centinela porque es igualmente imposible como hora de salida real (nadie sale a las 00:00 exacto en este contexto). Ambos son válidos — lo importante es la consistencia interna.

¿Qué pasa si `University.txt` no existe al iniciar? En Haskell, `doesFileExist` lo detecta y crea el archivo vacío antes de intentar leer. El programa arranca con lista vacía y funciona normalmente.

¿Por qué la Opción 4 recarga desde disco en Haskell? Porque la guía dice explícitamente: *"Choosing this option must: Load student information from University.txt"*. No dice "mostrar lo que está en memoria" — dice cargar desde el archivo.

> **¿Cómo garantizan que no haya dos Check In del mismo estudiante?** En Haskell, `checkIn` busca con `find` si ya existe un registro con ese ID y `salida = -1` antes de agregar. En Prolog, la cláusula tiene un guard: `estudiante(Id, _, 0) -> writeln('Error: ya está dentro')`.
>
> **Explica la diferencia entre `assertz` y `asserta`.** `assertz` agrega el hecho al **final** de la base de conocimiento, `asserta` al **inicio**. Usamos `assertz` para mantener el orden de inserción cronológico.

Checklist para la demostración en clase

Haskell

- [ ] Compilar/ejecutar: `runhaskell Main.hs`
- [ ] Check In con hora válida (ej: `08:30`)
- [ ] Check In duplicado → mensaje de error
- [ ] Search by ID → estudiante encontrado
- [ ] Search by ID → estudiante no encontrado
- [ ] Time Calculation → estudiante con salida
- [ ] List Students → recarga desde archivo
- [ ] Check Out → muestra tiempo calculado
- [ ] Verificar `University.txt` actualizado

Prolog

- [ ] Ejecutar: `swipl registro.pl`
- [ ] Check In con ID y hora
- [ ] Buscar estudiante activo
- [ ] Check Out → salida registrada
- [ ] Listar estudiantes
- [ ] Calcular tiempo de permanencia
- [ ] Verificar `University.txt` actualizado
- [ ] Salir limpiamente con opción 5